ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		В. В. Жукалин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №4

РЕАЛИЗАЦИЯ АДАПТИВНОСТИ КЛИЕНТСКОЙ ЧАСТИ ВЕБ-ПРИЛОЖЕНИЯ

по курсу: Web-программирование

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4321		И.А. Лаврешин
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Реализовать адаптивную клиентскую часть веб-приложения SignalBoard на Next.js: создать гибкие макеты для галереи, модального окна, бургер-меню и карты, настроить responsive-изображения, проверить приложение на нескольких ширинах viewport, выполнить Lighthouse-аудит и документированно устранить обнаруженные проблемы.

2 Задание

Продолжить проект лабораторной работы № 3. Проверить автоматический viewport metadata в App Router, применить mobile-first подход и обоснованные контрольные точки, использовать относительные размеры, CSS Grid, Flexbox, minmax() и clamp(), исключить горизонтальную прокрутку на узких экранах, настроить свойство sizes компонента next/image, адаптировать модальное окно к ширине и динамической высоте viewport, адаптировать бургер-меню с сохранением клавиатурной доступности, обновлять размеры карты после изменения контейнера, учесть hover, тип указателя и prefers-reduced-motion. Проверить интерфейс на ширинах 320, 375, 768, 1024 и 1440 CSS px и при масштабе 200 %, выполнить по три исходных и итоговых Lighthouse-запуска production-сборки с одинаковой конфигурацией, исправить не менее трёх подтверждённых проблем и проверить npm run lint и npm run build.

3 Ход выполнения задания

Исходное состояние. После лабораторной работы № 3 приложение содержало галерею сигналов с фотографиями, модальное окно, бургер-меню и карту, но размеры были заданы преимущественно в пикселях: сетка карточек minmax(240px, 1fr), карта фиксированной высоты 480px, модальное окно ограничивалось 100dvh без учёта альбомной ориентации, hover-эффект срабатывал и на сенсорных экранах, а изображения имели общее значение sizes.

Контрольные точки выбраны по содержимому, а не по моделям устройств. 30rem (480 px) – ширина, на которой модальному окну и карте становится тесно и нужно уменьшить отступы; 48rem (768 px) – с этой ширины в ряд помещаются две карточки минимальной ширины 16rem с отступами. Остальная перестройка выполняется без медиазапросов за счёт auto-fit, minmax() и clamp(). Базовые стили написаны для узкого экрана (mobile-first), медиазапросы min-width только добавляют возможности.

Viewport metadata. Next.js App Router автоматически добавляет в head тег <meta name="viewport" content="width=device-width, initial-scale=1">, поэтому вручную его писать не нужно. Масштабирование не отключено (нет maximum-scale и user-scalable=no). В layout.tsx через экспорт viewport задаются только цвет темы и цветовая схема:

```
export const viewport: Viewport = {
  themeColor: "#eef3f8",
  colorScheme: "light",
};
```

Адаптивная галерея. Одна и та же разметка перестраивается от одной до трёх-четырёх колонок без дублирования JSX. Функция min(100%, 15rem) не даёт карточке стать шире экрана 320 px, aspect-ratio резервирует место изображения до загрузки:

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 15rem), 1fr));
  gap: clamp(0.9rem, 2.5vw, 1.5rem);
}
.imageWrap { position: relative; aspect-ratio: 3 / 2; overflow: hidden; }
@media (hover: hover) and (pointer: fine) {
  .card:hover { transform: translateY(-0.15rem); }
```

```
}
@media (min-width: 48rem) {
  .grid { grid-template-columns: repeat(auto-fit, minmax(16rem, 1fr)); }
}
```

Свойство sizes сообщает браузеру реальную ширину слота изображения, чтобы из srcset выбирался подходящий файл. Для первой карточки, которая является LCP-изображением первого экрана, отключена ленивая загрузка и задан высокий приоритет:

```
<Image
  src={signal.image}
  alt={signal.imageAlt}
  fill
  sizes="(max-width: 47.99rem) calc(100vw - 2rem), (max-width: 74.99rem)
45vw, 22rem"
  className={styles.image}
  loading={index === 0 ? "eager" : "lazy"}
  fetchPriority={index === 0 ? "high" : "auto"}
/>
```

Адаптивное модальное окно помещается в динамическую высоту viewport 100dvh, которая учитывает появление и скрытие панели адреса мобильного браузера, и прокручивает длинное содержимое. Клавиатурная модель лабораторной работы № 3 сохранена:

```
.overlay { padding: clamp(0.5rem, 3vw, 1.5rem); }
.dialog {
  width: min(47.5rem, 100%);
  max-height: calc(100dvh - 1rem);
  overflow-y: auto;
  padding: clamp(1rem, 4vw, 1.75rem);
}
@media (orientation: landscape) and (max-height: 32rem) {
  .dialog { max-height: calc(100dvh - 0.5rem); }
}
```

Адаптивное бургер-меню занимает всю высоту 100dvh, учитывает вырезы экрана через env(safe-area-inset-*) и на ширине до 30rem раскрывается на весь экран:

```
.openButton {
  top: max(0.75rem, env(safe-area-inset-top));
  right: max(0.75rem, env(safe-area-inset-right));
}
.panel {
  width: min(26rem, 100%);
  height: 100dvh;
  overflow-y: auto;
  padding: max(1rem, env(safe-area-inset-top)) max(1rem, env(safe-area-inset-
right))
    max(1rem, env(safe-area-inset-bottom)) 1rem;
}
```

Карта. Высота контейнера задана через clamp(18rem, 58vh, 32rem), на узком экране – min(65dvh, 24rem). API Яндекс Карт само не узнаёт об

изменении размеров контейнера, поэтому ResizeObserver вызывает map.container.fitToViewport(), а в cleanup наблюдатель отключается:

```
resizeObserver = new ResizeObserver(() => {
  map?.container.fitToViewport();
});
resizeObserver.observe(containerRef.current);
// cleanup
return () => {
  disposed = true;
  resizeObserver?.disconnect();
  map?.destroy();
};
```

Режим уменьшенного движения задаётся в globals.css и отключает необязательные анимации и плавную прокрутку по системной настройке пользователя:

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    scroll-behavior: auto !important;
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
  }
}
```

Ручная матрица ширин. Production-сборка открыта в браузере с viewport 320, 375, 768, 1024 и 1440 CSS px; для каждого маршрута сравнивались scrollWidth и clientWidth корневого элемента и считалось число колонок сетки. Результаты приведены в таблице 1: горизонтальной прокрутки нет ни на одной ширине.

Таблица 1 – Ручная матрица контрольных ширин

Ширина	**Колонок /gallery**	**Колонок /signals**	**Гориз. прокрутка**	**Модальное окно, меню**
320 px	1	1	нет	Помещаются, фокус сохранён
375 px	1	1	нет	Помещаются, фокус сохранён
768 px	2	2	нет	Помещаются, фокус сохранён
1024 px	3	3	нет	Помещаются, фокус сохранён
1440 px	3	4	нет	Помещаются, фокус сохранён

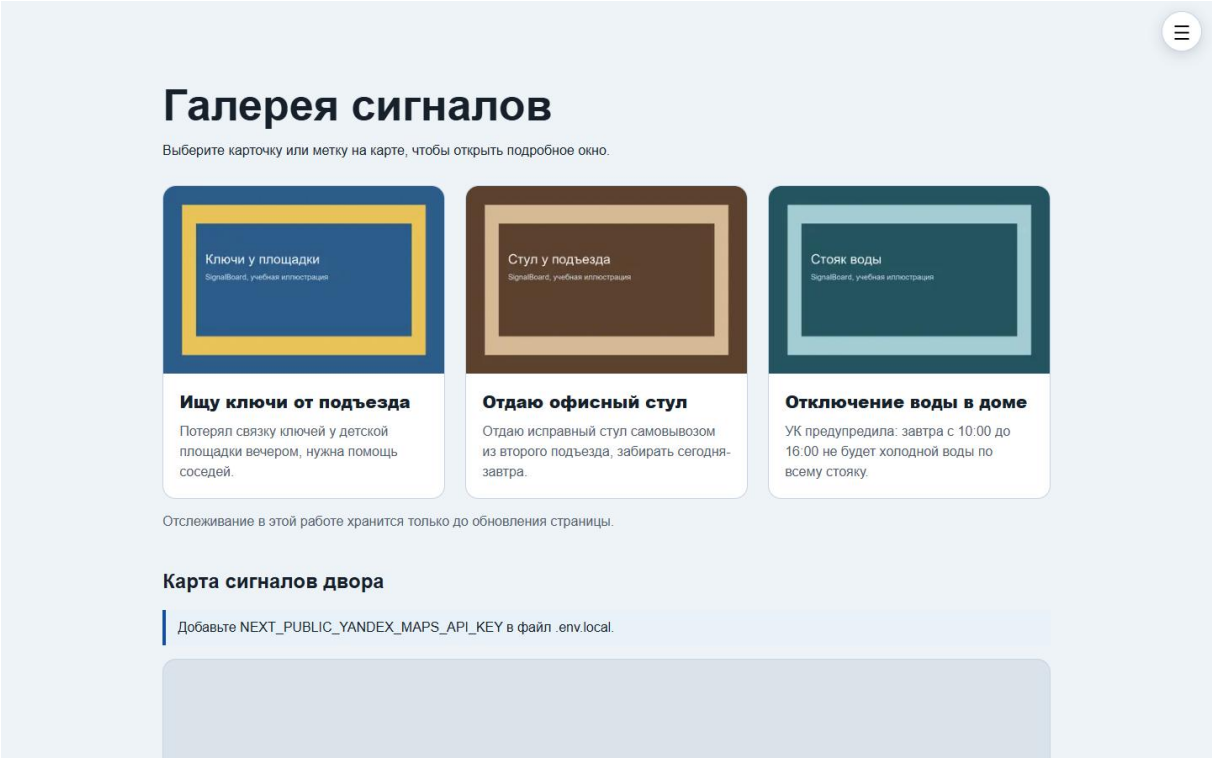


Рисунок 1 – Галерея на ширине 1440 CSS px

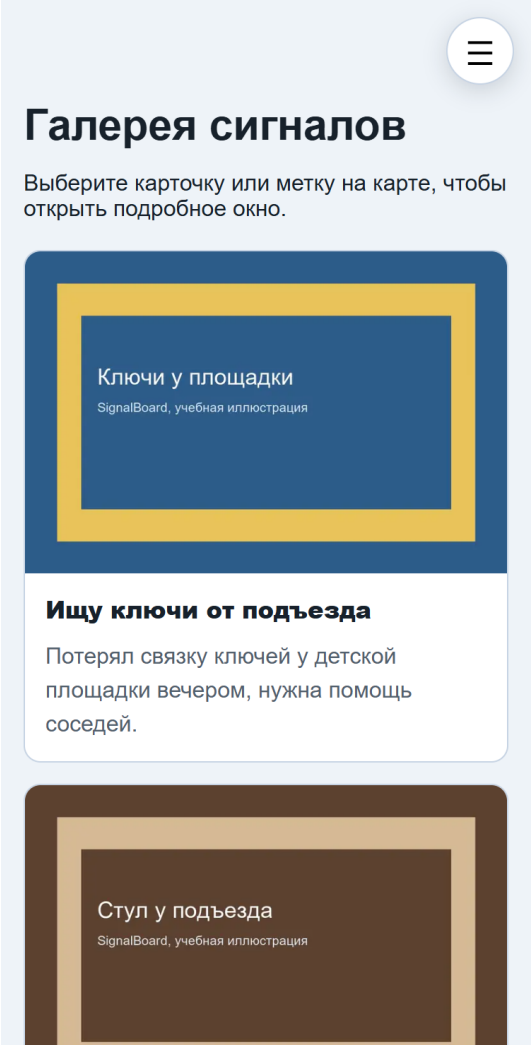


Рисунок 2 – Галерея на ширине 375 CSS px

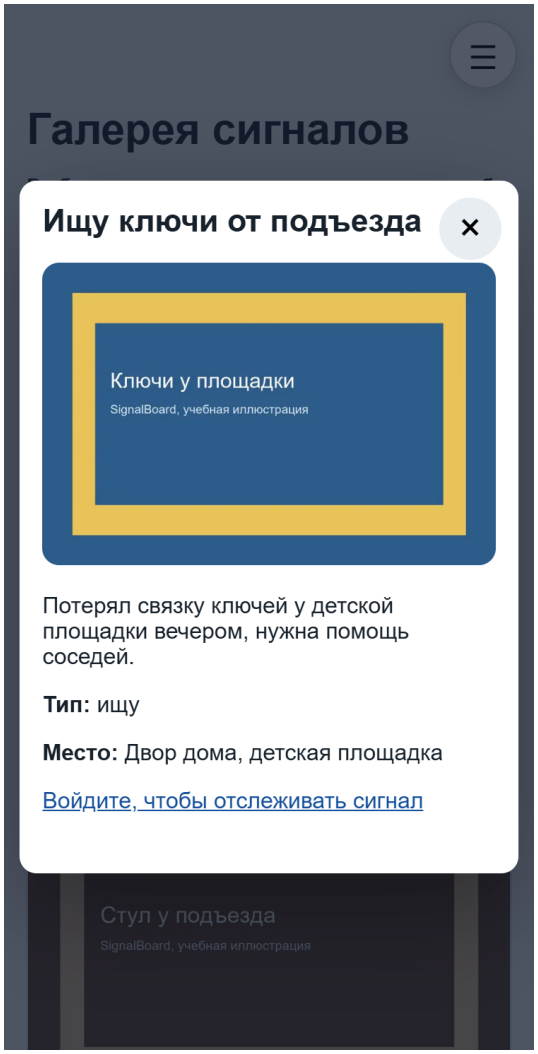


Рисунок 3 – Модальное окно на узком экране 375 CSS px

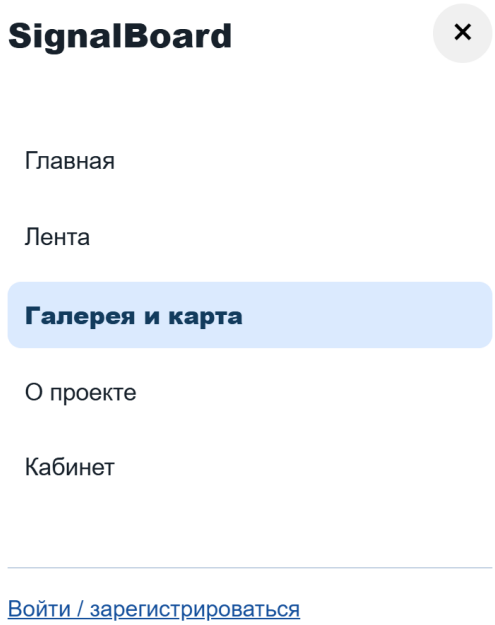


Рисунок 4 – Открытое бургер-меню на узком экране

Карта после изменения ширины. Окно браузера было сужено с 1440 до 800 CSS px без перезагрузки. Контейнер карты изменил размер вместе с сеткой, ResizeObserver сработал и вызвал fitToViewport. Без собственного ключа API Яндекс Карт не загружается, поэтому на снимке виден контейнер и сообщение о ключе. Снимок с загруженной картой выполняется после записи ключа в .env.local.

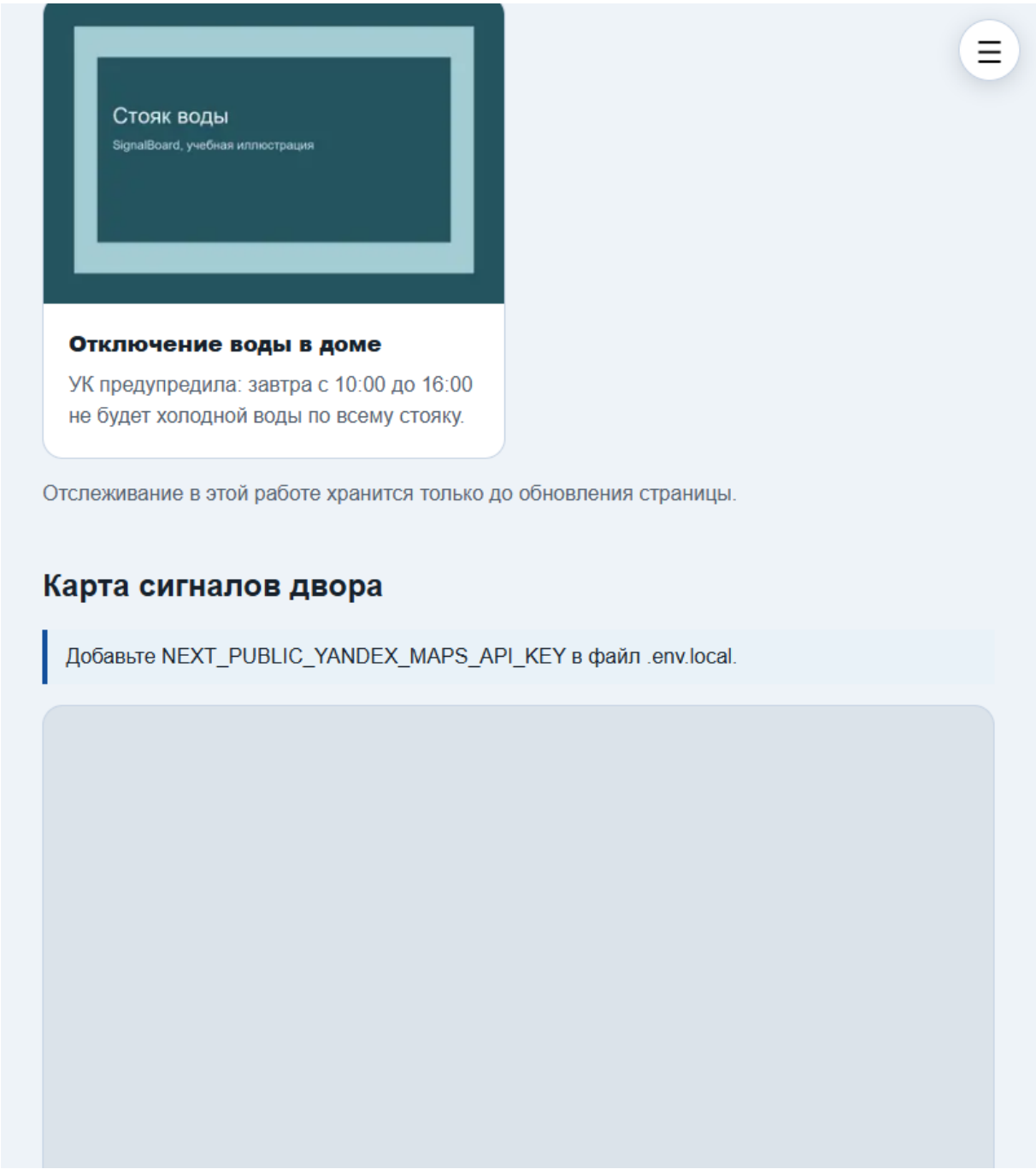


Рисунок 5 – Раздел карты после сужения окна до 800 CSS px

[ВСТАВИТЬ СКРИНШОТ: карта с ключом: сузить окно браузера и показать, что серых областей нет]

Рисунок 6 – Карта с метками после изменения ширины окна

Проверка масштаба 200 %. Масштаб 200 % при окне шириной 1280 px соответствует viewport 640 CSS px. На этой ширине scrollWidth = 640 и

clientWidth = 640: горизонтальной прокрутки нет, текст не обрезается, кнопка меню и карточки остаются доступными.

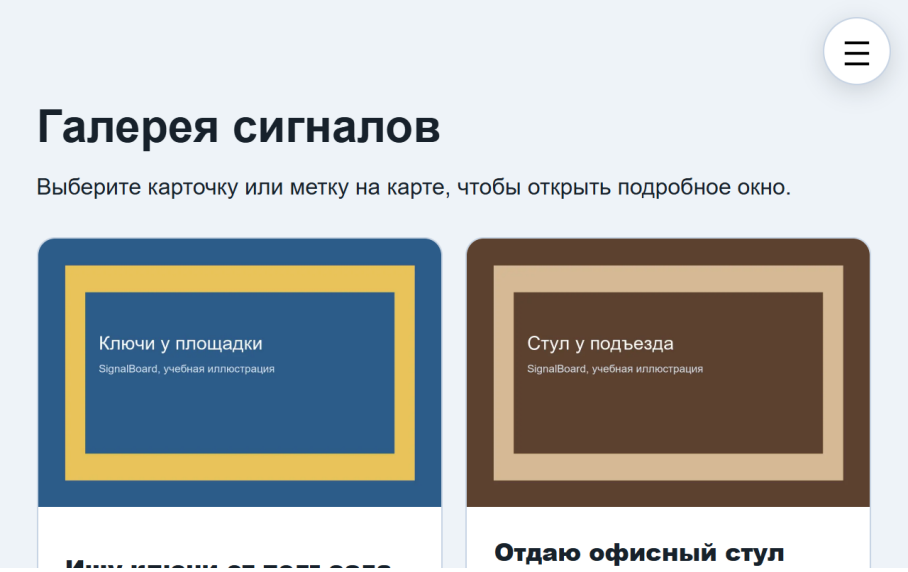


Рисунок 7 – Галерея при масштабе 200 %

Конфигурация Lighthouse. Lighthouse 12.8.2 (тот же движок, что во вкладке Lighthouse DevTools), браузер Google Chrome 154.0.8037.98, дата проверки 10.10.2026. Режим Navigation, устройство Mobile, троттлинг simulated (по умолчанию), категории Performance, Accessibility, Best Practices и SEO, адрес http://localhost:3004/gallery. Проверялась production-сборка (npm run build и npm run start), расширения браузера не участвовали (чистый headless-профиль). Конфигурация всех шести запусков одинакова.

Таблица 2 – Исходные результаты Lighthouse

Запуск	Perf.	Access.	Best Pr.	SEO	LCP	TBT	CLS
Исходный 1	99	95	100	100	2173 мс	36 мс	0
Исходный 2	99	95	100	100	2159 мс	36 мс	0
Исходный 3	99	95	100	100	2152 мс	33 мс	0
Медиана	99	95	100	100	2159 мс	36 мс	0

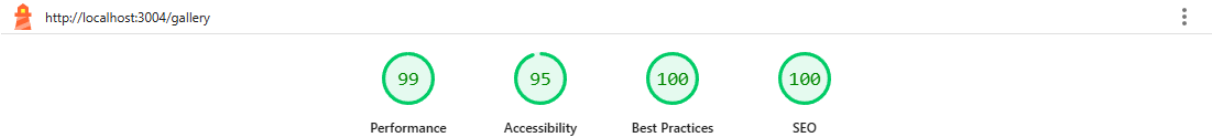


Рисунок 8 – Исходный Lighthouse-отчёт (медианный запуск)

Подтверждённые проблемы (failed audits), выбранные для исправления:

– aria-prohibited-attr (Accessibility) – у div-контейнера карты был атрибут aria-label без роли; для элемента без роли доступное имя запрещено и скринридер его не объявляет;

– lcp-lazy-loaded (Performance) – изображение первой карточки, являющееся Largest Contentful Paint, загружалось лениво (loading="lazy" по умолчанию у next/image) и запрашивалось позже необходимого;

– lcp-discovery-insight, пункт «fetchpriority=high should be applied» (Performance) – LCP-изображению не был назначен высокий приоритет загрузки.

Внесённые исправления:

– контейнеру карты добавлен role="region", после чего aria-label стал допустимым и карта объявляется как именованная область;

– для первой карточки галереи задан loading="eager", остальные карточки оставлены ленивыми – предзагрузка всех изображений конкурировала бы за сеть;

– для первой карточки задан fetchPriority="high", остальные получают "auto".

Аудиты render-blocking-resources, unused-javascript и legacy-javascript относятся к служебным CSS- и JS-чанкам самого фреймворка Next.js, а не к коду приложения; их исправление потребовало бы изменения конфигурации сборки, поэтому они зафиксированы как известные и не исправлялись.

Таблица 3 – Итоговые результаты Lighthouse

Запуск	**Perf.**	**Access.**	**Best Pr.**	**SEO**	**LCP**	**TBT**	**CLS**
Итоговый 1	99	100	100	100	2208 мс	30 мс	0
Итоговый 2	99	100	100	100	2152 мс	32 мс	0
Итоговый 3	99	100	100	100	2034 мс	35 мс	0
Медиана	99	100	100	100	2152 мс	32 мс	0

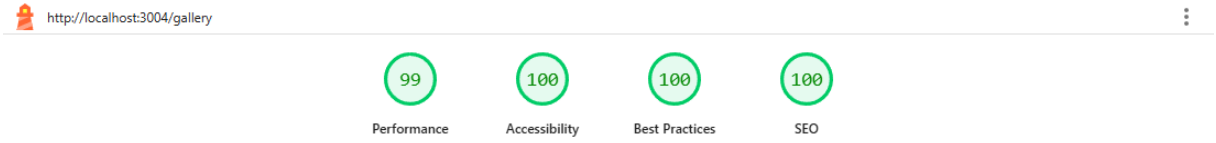


Рисунок 9 – Итоговый Lighthouse-отчёт (медианный запуск)

Таблица 4 – Сравнение медиан до и после исправлений

Показатель	**До**	**После**
Performance	99	99
Accessibility	95	100
Best Practices	100	100
SEO	100	100
LCP	2159 мс	2152 мс
TBT	36 мс	32 мс
CLS	0	0

Accessibility вырос с 95 до 100 за счёт устранения aria-prohibited-attr. Аудиты lcp-lazy-loaded и lcp-discovery-insight перестали быть failed. Performance остался 99: приложение небольшое и работает на локальном сервере, поэтому LCP около 2,2 с определяется в основном моделью simulated-троттлинга, а разница медиан LCP и TBT между сериями находится в пределах разброса запусков. CLS равен нулю благодаря aspect-ratio у областей изображений. Регрессий функциональности нет: модальное окно, меню и карта проверены вручную после исправлений.

Лабораторные и полевые данные. Lighthouse даёт лабораторные (lab) данные: один запуск на конкретной машине с эмуляцией устройства и сети. Полевые (field) данные собираются у реальных пользователей (например, Chrome UX Report) на разных устройствах и сетях и отражают 75-й перцентиль. Поэтому локальный аудит помогает найти и воспроизвести проблемы, но не заменяет полевые метрики; INP Lighthouse не измеряет вовсе, так как для него нужно реальное взаимодействие, и использует TBT как лабораторный индикатор отзывчивости.

Проверка кода. Команды npm run lint и npm run build выполнены после всех исправлений и завершились без ошибок (рисунки 10–11).

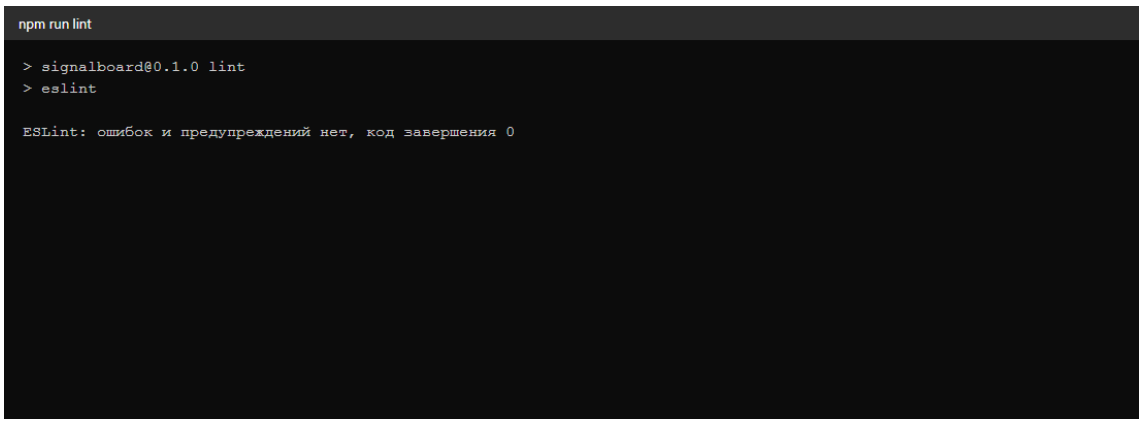


Рисунок 10 – Результат npm run lint

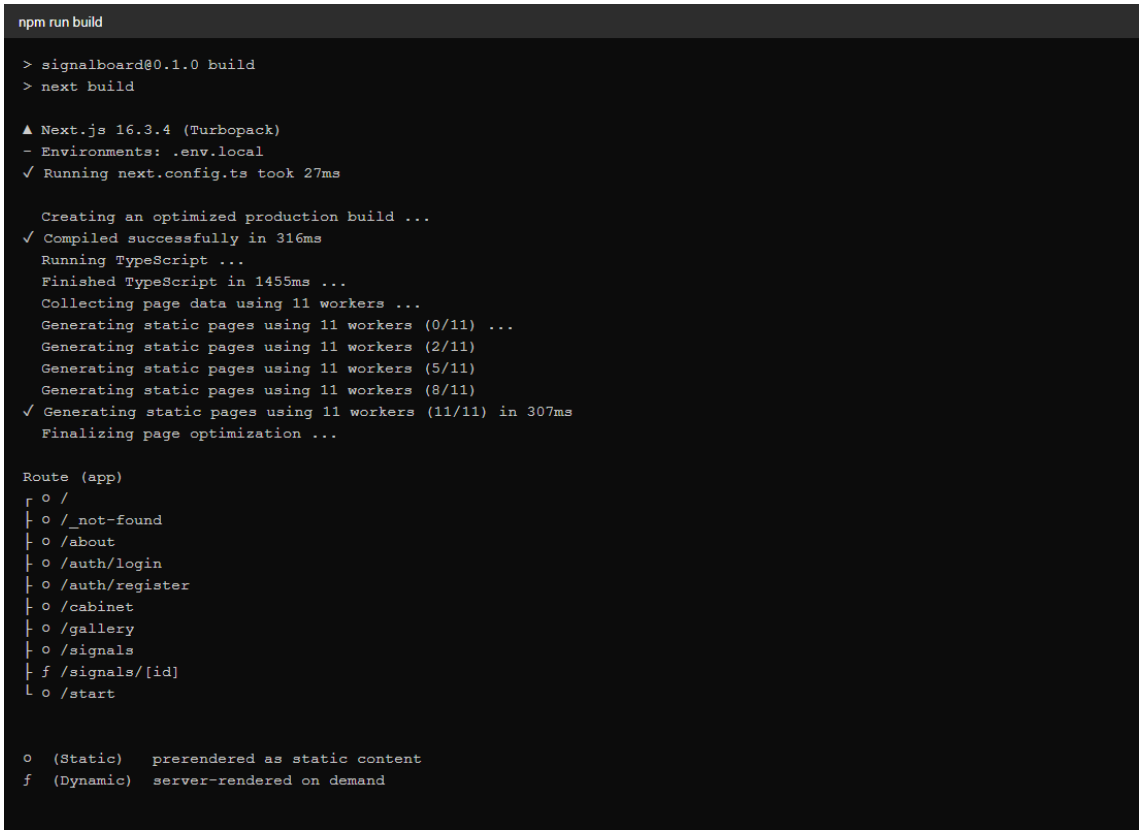


Рисунок 11 – Результат npm run build

4 Вывод

В ходе работы клиентская часть SignalBoard сделана адаптивной по принципу mobile-first: сетка карточек перестраивается через auto-fit и minmax() без дублирования разметки, размеры заданы через clamp() и относительные единицы, изображения получили корректный sizes и резерв пропорций, модальное окно и меню учитывают динамическую высоту 100dvh и безопасные области, карта пересчитывается через ResizeObserver и fitToViewport, hover включается только для точного указателя, учтён prefers-reduced-motion. Ручная проверка на пяти контрольных ширинах и при масштабе 200 % не выявила горизонтальной прокрутки. По результатам трёх исходных и трёх итоговых Lighthouse-запусков устранены три подтверждённые проблемы, медиана Accessibility выросла с 95 до 100. Проект проходит npm run lint и npm run build.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

Файл app/layout.tsx

```
// Настраивает метаданные, viewport, провайдер и общую оболочку приложения.

import type { Metadata, Viewport } from "next";
import type { ReactNode } from "react";
import DemoAuthProvider from "./providers/DemoAuthProvider";
import BurgerMenu from "./ui/BurgerMenu/BurgerMenu";
import "./globals.css";

// Задаём заголовок и описание страницы для Metadata API.
export const metadata: Metadata = {
  title: "SignalBoard",
  description: "Доска локальных сигналов соседям",
};

// Указываем цвет темы и цветовую схему viewport.
export const viewport: Viewport = {
  themeColor: "#eef3f8",
  colorScheme: "light",
};

// Объявляем основной компонент этого файла.
export default function RootLayout({
  children,
}: Readonly<{ children: ReactNode }>) {
  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <html lang="ru">
      <body>
        <DemoAuthProvider>
          <BurgerMenu />
          {children}
        </DemoAuthProvider>
      </body>
    </html>
  );
}
```

Файл app/globals.css

```
/* Задаёт дизайн-токены, базовую адаптивную страницу и режим уменьшенного
движения. */

/* Объявляем общие дизайн-токены приложения. */
:root {
  --page-max: 72rem;
  --page-inline: clamp(1rem, 4vw, 3rem);
  --surface: #ffffff;
  --background: #eef3f8;
  --text: #17212b;
  --muted: #52606d;
  --border: #c7d4e4;
  --accent: #134f9c;
  --focus: #005fcc;
}

*,
```

```
*::before,
*::after {
  box-sizing: border-box;
}

html {
  color-scheme: light;
  background: var(--background);
}

/* Задаём базовые параметры страницы и типографики. */
body {
  min-width: 20rem;
  margin: 0;
  background: var(--background);
  color: var(--text);
  font-family: Arial, Helvetica, sans-serif;
}

/* Ограничиваем ширину основного содержимого и выравниваем его по центру. */
main {
  width: min(var(--page-max), 100%);
  margin-inline: auto;
  padding: clamp(4.5rem, 8vw, 6rem) var(--page-inline) 4rem;
}

h1 {
  margin-block: 0 1rem;
  font-size: clamp(1.9rem, 5vw, 3.25rem);
  line-height: 1.1;
  text-wrap: balance;
}

img,
svg,
video,
canvas {
  max-width: 100%;
}

button,
a,
input,
select,
textarea {
  font: inherit;
}

a {
  color: var(--accent);
}

button {
  cursor: pointer;
}

/* Показываем заметный фокус при клавиатурной навигации. */
:focus-visible {
  outline: 0.25rem solid var(--focus);
  outline-offset: 0.2rem;
}
```

```
.eyebrow {
  margin-bottom: 8px;
  font-weight: 800;
  letter-spacing: 0.08em;
  text-transform: uppercase;
}

.lead {
  max-width: 680px;
  font-size: clamp(1.05rem, 2vw, 1.25rem);
  line-height: 1.6;
}

.primary-link {
  display: inline-block;
  margin-top: 16px;
  padding: 12px 18px;
  border-radius: 10px;
  background: var(--accent);
  color: #ffffff;
  font-weight: 700;
  text-decoration: none;
}

.feature-list,
.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 15rem), 1fr));
  gap: clamp(0.9rem, 2.5vw, 1.5rem);
}

.feature-card,
.card {
  min-width: 0;
  padding: 22px;
  border: 1px solid var(--border);
  border-radius: 14px;
  background: var(--surface);
}

.form-page {
  max-width: 620px;
}

.auth-form {
  display: grid;
  gap: 10px;
  margin: 24px 0;
  padding: 24px;
  border-radius: 14px;
  background: var(--surface);
}

.auth-form input {
  min-height: 44px;
  padding: 8px 10px;
  border: 1px solid #8d99a6;
  border-radius: 8px;
}
```

```
.auth-form button {
  min-height: 44px;
  margin-top: 10px;
  border: 0;
  border-radius: 8px;
  background: var(--accent);
  color: #ffffff;
  font-weight: 700;
}

.demo-warning {
  padding: 12px 14px;
  border-left: 4px solid #c67c00;
  background: #fff4d6;
}

.form-error {
  color: #9b1c1c;
  font-weight: 700;
}

.profile-list {
  display: grid;
  grid-template-columns: minmax(4rem, 6.25rem) 1fr;
  gap: 8px 16px;
  max-width: 520px;
}

.profile-list dt {
  font-weight: 700;
}

.profile-list dd {
  margin: 0;
  overflow-wrap: anywhere;
}

/* Уменьшаем необязательное движение по системной настройке пользователя. */
@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    scroll-behavior: auto !important;
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
  }
}
```

Файл app/gallery/_components/SignalsGallery.tsx

```
// Управляет карточками, выбранным объектом, отслеживанием, модальным окном и
картой.

"use client";

import Image from "next/image";
import { useCallback, useState } from "react";
import { neighborSignals, type NeighborSignal } from "../../data/signals";
import PhotoMap from "./PhotoMap";
import PhotoModal from "./PhotoModal";
import styles from "./SignalsGallery.module.css";
```

```
// Объявляем основной компонент этого файла.
export default function SignalsGallery() {
  // Создаём локальное состояние компонента.
  const [selected, setSelected] = useState<NeighborSignal | null>(null);
  const [trackedIds, setTrackedIds] = useState<Set<string>>(
    () => new Set<string>(),
  );
  const [returnFocusTo, setReturnFocusTo] = useState<HTMLButtonElement |
null>(null);

  function openFromCard(signal: NeighborSignal, button: HTMLButtonElement) {
    setReturnFocusTo(button);
    setSelected(signal);
  }

  const openFromMap = useCallback((signal: NeighborSignal) => {
    setReturnFocusTo(null);
    setSelected(signal);
  }, []);

  function closeModal() {
    setSelected(null);
  }

  function toggleTracked(id: string) {
    setTrackedIds((current) => {
      const next = new Set(current);
      if (next.has(id)) next.delete(id);
      else next.add(id);
      return next;
    });
  }

  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <>
      <div className={styles.grid}>
        {neighborSignals.map((signal, index) => (
          <button
            className={styles.card}
            type="button"
            key={signal.id}
            onClick={(event) => openFromCard(signal, event.currentTarget)}
          >
            <span className={styles.imageWrap}>
              <Image
                src={signal.image}
                alt={signal.imageAlt}
                fill
                sizes="(max-width: 47.99rem) calc(100vw - 2rem), (max-width:
74.99rem) 45vw, 22rem"
                className={styles.image}
                // Первая карточка является LCP-изображением первого экрана.
                loading={index === 0 ? "eager" : "lazy"}
                fetchPriority={index === 0 ? "high" : "auto"}
              />
            </span>
            <span className={styles.cardBody}>
              <span className={styles.title}>{signal.title}</span>
              <span className={styles.summary}>{signal.summary}</span>
            </span>
```

```
          </button>
        ))}
      </div>

      <p className={styles.demoNote}>
        Отслеживание в этой работе хранится только до обновления страницы.
      </p>

      <PhotoMap signals={neighborSignals} onSelect={openFromMap} />

      {selected ? (
        <PhotoModal
          signal={selected}
          isTracked={trackedIds.has(selected.id)}
          onToggleTracked={() => toggleTracked(selected.id)}
          onClose={closeModal}
          returnFocusTo={returnFocusTo}
        />
      ) : null}
    </>
  );
}
```

Файл app/gallery/_components/SignalsGallery.module.css

```
/* Перестраивает галерею, резервирует место изображения и добавляет
безопасный hover. */

/* Создаём адаптивную сетку карточек. */
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 15rem), 1fr));
  gap: clamp(0.9rem, 2.5vw, 1.5rem);
  margin-top: clamp(1.25rem, 4vw, 2rem);
}

/* Оформляем интерактивную карточку объекта. */
.card {
  display: grid;
  min-width: 0;
  overflow: hidden;
  padding: 0;
  border: 1px solid var(--border);
  border-radius: clamp(0.75rem, 2vw, 1rem);
  background: var(--surface);
  color: inherit;
  text-align: left;
  cursor: pointer;
  transition: transform 160ms ease, box-shadow 160ms ease;
}

/* Резервируем область изображения с постоянными пропорциями. */
.imageWrap {
  position: relative;
  display: block;
  aspect-ratio: 3 / 2;
  overflow: hidden;
  background: #d9dde2;
}

.image {
  object-fit: cover;
```

```
}

.cardBody {
  display: grid;
  gap: 0.6rem;
  padding: clamp(0.9rem, 3vw, 1.2rem);
}

.title,
.summary {
  min-width: 0;
  overflow-wrap: anywhere;
}

.title {
  font-size: clamp(1.05rem, 2vw, 1.25rem);
  font-weight: 800;
}

.summary {
  color: var(--muted);
  line-height: 1.5;
}

.demoNote {
  margin: 18px 0 32px;
  color: var(--muted);
}

/* Включаем hover-эффект только для точного указателя. */
@media (hover: hover) and (pointer: fine) {
  .card:hover {
    transform: translateY(-0.15rem);
    box-shadow: 0 0.8rem 1.8rem rgb(15 23 42 / 14%);
  }
}

/* Меняем раскладку на соответствующей контрольной ширине. */
@media (min-width: 48rem) {
  .grid {
    grid-template-columns: repeat(auto-fit, minmax(16rem, 1fr));
  }
}
```

Файл app/gallery/_components/PhotoModal.module.css

```
/* Адаптирует модальное окно к ширине, высоте и ориентации viewport. */

/* Перекрываем viewport затемнённым фоном. */
.overlay {
  position: fixed;
  inset: 0;
  z-index: 3000;
  display: grid;
  place-items: center;
  padding: clamp(0.5rem, 3vw, 1.5rem);
  background: rgb(15 23 42 / 72%);
}

/* Ограничиваем размеры окна и разрешаем прокрутку длинного содержимого. */
.dialog {
  position: relative;
```

```
  width: min(47.5rem, 100%);
  max-height: calc(100dvh - 1rem);
  overflow-y: auto;
  overscroll-behavior: contain;
  padding: clamp(1rem, 4vw, 1.75rem);
  border-radius: clamp(0.75rem, 2vw, 1rem);
  background: var(--surface);
  box-shadow: 0 1.5rem 4.4rem rgb(0 0 0 / 35%);
}

.dialog h2 {
  margin: 0 3rem 1rem 0;
  font-size: clamp(1.4rem, 4vw, 2rem);
  overflow-wrap: anywhere;
}

.close {
  position: absolute;
  top: 0.75rem;
  right: 0.75rem;
  display: grid;
  width: 2.75rem;
  height: 2.75rem;
  place-items: center;
  border: 0;
  border-radius: 50%;
  background: #e8edf2;
  font-size: 1.75rem;
  line-height: 1;
}

.image {
  width: 100%;
  height: auto;
  border-radius: 0.75rem;
  object-fit: cover;
}

.actions {
  display: flex;
  flex-wrap: wrap;
  gap: 0.75rem;
  margin-top: 1rem;
}

.actions > * {
  min-height: 2.75rem;
}

/* Меняем раскладку на соответствующей контрольной ширине. */
@media (max-width: 30rem) {
  .actions {
    display: grid;
  }

  .actions > * {
    width: 100%;
  }
}

/* Учитываем небольшую высоту экрана в альбомной ориентации. */
```

```
@media (orientation: landscape) and (max-height: 32rem) {
  .dialog {
    max-height: calc(100dvh - 0.5rem);
  }
}
```

Файл app/gallery/_components/PhotoMap.tsx

```
// Загружает API Яндекс Карт, создаёт карту и связывает метки с объектами.

"use client";

import { useEffect, useRef, useState } from "react";
import type { NeighborSignal } from "../../data/signals";
import styles from "./PhotoMap.module.css";

// Координаты представлены парой широты и долготы.
type Coordinates = [number, number];

type YandexPlacemark = {
  events: {
    add: (eventName: string, handler: () => void) => void;
  };
};

type YandexMap = {
  container: {
    fitToViewport: () => void;
  };
  geoObjects: {
    add: (placemark: YandexPlacemark) => void;
  };
  destroy: () => void;
};

type YandexMapsApi = {
  ready: (callback: () => void) => void;
  Map: new (
    container: HTMLElement,
    state: { center: Coordinates; zoom: number; controls: string[] },
  ) => YandexMap;
  Placemark: new (
    coordinates: Coordinates,
    properties: { hintContent: string; balloonContent: string },
    options: { preset: string; iconColor: string },
  ) => YandexPlacemark;
};

// Дополняем тип Window объектом, который добавляет внешний API.
declare global {
  interface Window {
    ymaps?: YandexMapsApi;
  }
}

const scriptId = "signalboard-yandex-maps";
let mapsPromise: Promise<YandexMapsApi> | null = null;

// Загружаем скрипт карты один раз и ожидаем готовности API.
function loadMaps(apiKey: string): Promise<YandexMapsApi> {
  if (window.ymaps) {
    return new Promise((resolve) => {
```

```
      window.ymaps?.ready(() => resolve(window.ymaps as YandexMapsApi));
    });
  }

  if (mapsPromise) return mapsPromise;

  mapsPromise = new Promise((resolve, reject) => {
    function resolveWhenReady() {
      if (!window.ymaps) {
        mapsPromise = null;
        reject(new Error("Yandex Maps API is unavailable"));
        return;
      }

      window.ymaps.ready(() => resolve(window.ymaps as YandexMapsApi));
    }

    const existing = document.getElementById(scriptId) as HTMLScriptElement |
null;
    if (existing) {
      existing.addEventListener("load", resolveWhenReady, { once: true });
      existing.addEventListener("error", () => reject(new Error("Map script
error")), {
        once: true,
      });
      return;
    }

    const script = document.createElement("script");
    script.id = scriptId;
    script.src = `https://api-
maps.yandex.ru/2.1/?apikey=${encodeURIComponent(apiKey)}&lang=ru_RU`;
    script.async = true;
    script.addEventListener("load", resolveWhenReady, { once: true });
    script.addEventListener(
      "error",
      () => {
        mapsPromise = null;
        reject(new Error("Map script error"));
      },
      { once: true },
    );
    // Подключаем внешний скрипт к документу.
    document.head.appendChild(script);
  });

  return mapsPromise;
}

// Объявляем основной компонент этого файла.
export default function PhotoMap({
  signals,
  onSelect,
}: {
  signals: NeighborSignal[];
  onSelect: (signal: NeighborSignal) => void;
}) {
  const containerRef = useRef<HTMLDivElement>(null);
  // Создаём локальное состояние компонента.
  const apiKey = process.env.NEXT_PUBLIC_YANDEX_MAPS_API_KEY ?? "";
```

```
  const [status, setStatus] = useState<"loading" | "ready" | "error" | "no-
key">(
    apiKey ? "loading" : "no-key",
  );

  // Синхронизируем компонент с браузерными событиями или внешним API.
  useEffect(() => {
    if (!apiKey) {
      return;
    }

    if (!containerRef.current || signals.length === 0) return;

    let disposed = false;
    let map: YandexMap | null = null;
    let resizeObserver: ResizeObserver | null = null;

    async function buildMap() {
      try {
        const ymaps = await loadMaps(apiKey);
        if (disposed || !containerRef.current) return;

        const first = signals[0];
        // Создаём экземпляр карты в подготовленном DOM-контейнере.
        map = new ymaps.Map(containerRef.current, {
          center: [first.latitude, first.longitude],
          zoom: 15,
          controls: ["zoomControl"],
        });

        // Добавляем на карту метку для каждого сигнала.
        signals.forEach((signal) => {
          // Создаём метку с координатами и данными текущего объекта.
          const placemark = new ymaps.Placemark(
            [signal.latitude, signal.longitude],
            {
              hintContent: signal.title,
              balloonContent: signal.location,
            },
            {
              preset: "islands#circleDotIcon",
              iconColor: "#134f9c",
            },
          );

          placemark.events.add("click", () => onSelect(signal));
          map?.geoObjects.add(placemark);
        });

        // Следит за размером контейнера и уведомляет карту об изменении
viewport.
        if (containerRef.current && typeof ResizeObserver !== "undefined") {
          // Создаём наблюдатель за изменением размеров контейнера.
          resizeObserver = new ResizeObserver(() => {
            // Просим API карты пересчитать доступную область.
            map?.container.fitToViewport();
          });
          // Начинаем наблюдение за DOM-контейнером карты.
          resizeObserver.observe(containerRef.current);
        }
```

```
        setStatus("ready");
      } catch {
        if (!disposed) setStatus("error");
      }
    }

    void buildMap();

    // Освобождает наблюдатель и экземпляр карты при размонтировании
компонента.
    return () => {
      disposed = true;
      // Отключаем наблюдатель размеров.
      resizeObserver?.disconnect();
      // Уничтожаем экземпляр карты и его обработчики.
      map?.destroy();
    };
  }, [apiKey, onSelect, signals]);

  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <section className={styles.section} aria-labelledby="map-title">
      <h2 id="map-title">Карта сигналов двора</h2>

      {status === "no-key" ? (
        <p className={styles.status}>
          Добавьте NEXT_PUBLIC_YANDEX_MAPS_API_KEY в файл .env.local.
        </p>
      ) : null}

      {status === "error" ? (
        <p className={styles.status}>Не удалось загрузить карту.</p>
      ) : null}

      {status === "loading" ? (
        <p className={styles.status}>Карта загружается…</p>
      ) : null}

      <div
        ref={containerRef}
        className={styles.canvas}
        role="region"
        aria-label="Интерактивная карта локальных сигналов"
      />
    </section>
  );
}
```

Файл app/gallery/_components/PhotoMap.module.css

```
/* Задаёт адаптивную высоту карты и оформление сообщений состояния. */

.section {
  margin-top: clamp(2rem, 6vw, 3rem);
}

/* Задаём явные размеры контейнера интерактивной карты. */
.canvas {
  width: 100%;
  height: clamp(18rem, 58vh, 32rem);
  overflow: hidden;
  border: 1px solid var(--border);
```

```
  border-radius: clamp(0.75rem, 2vw, 1rem);
  background: #dbe3ea;
}

/* Выделяем сообщения о загрузке или ошибке карты. */
.status {
  padding: 0.75rem 0.9rem;
  border-left: 0.25rem solid var(--accent);
  background: #e9f2f8;
  overflow-wrap: anywhere;
}

/* Меняем раскладку на соответствующей контрольной ширине. */
@media (max-width: 30rem) {
  .canvas {
    height: min(65dvh, 24rem);
    border-radius: 0.6rem;
  }
}
```

Файл app/ui/BurgerMenu/styles.module.css

```
/* Адаптирует кнопку и панель бургер-меню с учётом safe area. */

/* Закрепляем кнопку открытия меню поверх страницы. */
.openButton {
  position: fixed;
  top: max(0.75rem, env(safe-area-inset-top));
  right: max(0.75rem, env(safe-area-inset-right));
  z-index: 2000;
  display: grid;
  width: 3rem;
  height: 3rem;
  place-items: center;
  border: 1px solid var(--border);
  border-radius: 50%;
  background: var(--surface);
  font-size: 1.5rem;
  cursor: pointer;
  pointer-events: auto;
  touch-action: manipulation;
  box-shadow: 0 4px 16px rgb(15 23 42 / 16%);
}

/* Скрываем кнопку открытия, пока панель уже на экране: иначе × стоит в тех
же пикселях. */
.openButton[aria-expanded="true"] {
  visibility: hidden;
  pointer-events: none;
}

/* Перекрываем viewport затемнённым фоном. */
.overlay {
  position: fixed;
  inset: 0;
  z-index: 2200;
  display: flex;
  justify-content: flex-end;
  background: rgb(15 23 42 / 55%);
}

/* Размещаем панель меню у правого края viewport. */
```

```
.panel {
  width: min(26rem, 100%);
  height: 100dvh;
  margin-left: auto;
  overflow-y: auto;
  overscroll-behavior: contain;
  padding: max(1rem, env(safe-area-inset-top))
    max(1rem, env(safe-area-inset-right))
    max(1rem, env(safe-area-inset-bottom))
    1rem;
  background: var(--surface);
  box-shadow: -1.25rem 0 3.1rem rgb(0 0 0 / 25%);
}

.header,
.authRow {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
}

.header a {
  color: var(--text);
  font-size: 24px;
  font-weight: 800;
  text-decoration: none;
}

.header button {
  width: 42px;
  height: 42px;
  border: 0;
  border-radius: 50%;
  font-size: 28px;
}

.list {
  display: grid;
  gap: 0.6rem;
  margin: clamp(2rem, 8vh, 3rem) 0;
  padding: 0;
  list-style: none;
}

.list a {
  display: block;
  min-height: 2.75rem;
  padding: 0.75rem;
  border-radius: 0.6rem;
  color: inherit;
  font-size: clamp(1.05rem, 4vw, 1.25rem);
  text-decoration: none;
}

.list a[aria-current="page"] {
  background: #dbeafe;
  color: #123b5d;
  font-weight: 800;
}
```

```
.authRow {
  padding-top: 20px;
  border-top: 1px solid var(--border);
}

/* Меняем раскладку на соответствующей контрольной ширине. */
@media (max-width: 30rem) {
  .panel {
    width: 100%;
  }

  .authRow {
    align-items: stretch;
    flex-direction: column;
  }
}
```